4.4 Summary

- A Bayesian optimization policy uses the trained Gaussian process to score how valuable each data point is in our quest of finding the optimum of the objective function. The score computed by the policy is called the acquisition score.
- In each iteration of a Bayesian optimization loop, a Gaussian process is trained on the observed data, a policy suggests a new data point to query, and the label of this point is added to the training set. This repeats until we cannot make more function evaluations.
- A GPyTorch model only needs minimal modifications in order to be integrated into BoTorch, which implements Bayesian optimization policies.
- BoTorch provides a helper function named `optimize_acqf()` from the `optim.optimize` module that takes in a policy object and returns the data point that maximizes the acquisition score.
- A good Bayesian optimization policy needs to balance between exploration (learning about regions with high uncertainty) and exploitation (narrowing down high-performing regions).
- Different Bayesian optimization policies address the exploration–exploitation tradeoff differently. It is important to inspect optimization progress in order to analyze and adjust the performance of the policy in use.
- One heuristic that could be used in Bayesian optimization is to find points that improve from the best-seen value.
- Finding the point that gives the best chance of improving from the best-seen value results in the *Probability of Improvement* policy.
- Finding the point that gives the highest expected improvement from the best-seen value results in the *Expected Improvement* policy.
- Probability of Improvement may be considered as an overly exploitative and risk-averse policy, as the policy only aims to improve from the best-seen value, however small the improvement it. Without any further modification, Expected Improvement tends to balance exploration and exploitation better than Probability of Improvement.
- Thanks to the Gaussian belief about the function values, computing scores by Probability of Improvement and Expected Improvement may be done in closed form. We therefore can compute and optimize the scores defined by these policies easily.

4.5 Exercise 2: Bayesian optimization for hyperparameter tuning

This exercise, implemented in `CH04/03 - Exercise 2.ipynb` applies BayesOpt to an objective function that simulates the accuracy surface of a support-vector machine model in a hyperparameter tuning task. The x-axis denotes the value of the penalty parameter C, while the y-axis denotes the value for the RBF kernel parameter γ. See the exercise in Chapter 3 for more detail. There are multiple steps:

1. Recreate the Bayesian optimization (BayesOpt) loop in `CH04/01 - BayesOpt loop.ipynb`.
   A. We don't need the Forrester function anymore; instead, copy the code for the two-dimensional function described in the exercise of Chapter 3 and use it as our objective function.

©Manning Publications Co. To comment go to liveBook
https://livebook.manning.com/#!/book/bayesian-optimization-in-action/discussion

Licensed to ashwin sampath <ashwin.manning@gmail.com>